

Tema 05 · Servidor web y JSP básico (scriptlets y formularios)

Clase 019 (26-08) · Teoría: «Web Server» (Clase019-JS-WebServer.pdf) y «JSP» (Clase020_jsp.pdf)

Programación Avanzada · Prof. Macrino Matías · Universidad del Salvador – Sede Pilar · 2.º cuatrimestre

1. El servidor web y el protocolo HTTP

Un **servidor web** es un programa que implementa el protocolo **HTTP** (HyperText Transfer Protocol) y está siempre escuchando peticiones de los clientes (navegadores). El navegador pide un recurso, el servidor responde con el contenido (HTML, imagen, JSON...). La palabra «servidor» designa tanto al programa como a la máquina donde corre. Ejemplos: Apache HTTP, Nginx, IIS y, para Java, **Apache Tomcat** (un contenedor web de Servlets/JSP).

Mensajes HTTP

Mensaje	Partes
Petición (request)	1) línea inicial con el método (GET, POST...) y la URL; 2) cabeceras (pares clave/valor); 3) cuerpo (parámetros o archivos que se envían)
Respuesta (response)	1) línea inicial con el código de estado ; 2) cabeceras (Content-Type, Set-Cookie...); 3) cuerpo (el HTML u otro contenido)

Métodos HTTP

Método	Para qué	Notas
GET	Leer un recurso	Solo consulta; los parámetros viajan en la URL; se puede repetir sin efectos
POST	Enviar datos que crean o modifican algo	Los datos viajan en el cuerpo; típico de formularios, altas y login
PUT	Actualizar (o crear) un recurso	No se usa desde formularios HTML
DELETE	Eliminar un recurso	Idem

Códigos de estado

Rango	Significado	Ejemplos
1xx	Informativo	100 Continue
2xx	Éxito	200 OK , 201 Created
3xx	Redirección	302 Found (sendRedirect)
4xx	Error del cliente	400 Bad Request, 403 Forbidden, 404 Not Found
5xx	Error del servidor	500 Internal Server Error

2. JSP: Java Server Pages

JSP es una tecnología **del lado del servidor** para generar páginas web dinámicas. Un archivo .jsp es HTML con código Java embebido. Cuando el usuario lo pide, el servidor **ejecuta el Java** y devuelve al

navegador solo el **HTML resultante** (el cliente nunca ve el código Java). Los .jsp se ubican en la raíz del proyecto web, al mismo nivel que la carpeta **WEB-INF** (lo que está dentro de WEB-INF no es accesible directamente desde el navegador).

Cómo funciona por dentro: la primera vez que se pide, Tomcat **traduce** el JSP a una clase Servlet Java, la compila y la ejecuta; las siguientes veces reutiliza la clase compilada. Es decir: **un JSP es un servlet disfrazado**.

Los tres tipos de código JSP

Tipo	Sintaxis	Para qué
Expresión	<code><%= expresión %></code>	Una sola expresión Java cuyo resultado se imprime en la página
Scriptlet	<code><% varias líneas de Java %></code>	Bloques de código: variables, if, for...
Declaración	<code><%! declaración %></code>	Declarar variables o métodos de la clase generada
Directiva	<code><%@ page ... %></code>	Configuración de la página: import, contentType (codificación), etc.
Comentario JSP	<code><%-- ... --%></code>	No llega al navegador (a diferencia del comentario HTML <code><!-- --></code> , que sí se envía)

Buena práctica: poner mucho código Java dentro del JSP es una mala práctica (mezcla presentación con lógica). Por eso más adelante se reemplazan los scriptlets por **JSTL** y la lógica se lleva a clases Java (servlets, DAO) — Temas 07 y 08. El parcial **pide JSTL**.

Objetos implícitos

En un JSP existen variables ya creadas por el contenedor, que se usan sin declararlas:

Objeto	Corresponde a	Uso típico
request	HttpServletRequest	Leer los datos del formulario: <code>request.getParameter("nombre")</code>
response	HttpServletResponse	Enviar cookies, redirigir: <code>response.sendRedirect("x.jsp")</code>
out	PrintWriter (JspWriter)	Escribir en la respuesta: <code>out.println("...")</code>
session	HttpSession	Guardar datos del usuario entre pantallas (Tema 06)
application	ServletContext	Datos compartidos por toda la aplicación
config / pageContext	ServletConfig / PageContext	Configuración y acceso a los demás objetos

3. Formularios: del HTML al JSP

El flujo básico de la web dinámica: el usuario completa un formulario HTML (formulario.html) → al enviar, el navegador manda una petición POST a la URL de action (procesar-form.jsp) con cada campo identificado por su **name** → el JSP lee los valores con **request.getParameter("name")** (que siempre devuelve **String**; hay que convertirlo con Integer.parseInt si es un número) → genera la respuesta HTML.

4. Maven y despliegue

El proyecto es de tipo **war** (Web ARchive). Se construye con mvn package y el archivo .war se copia a la carpeta webapps de Tomcat (o se ejecuta desde el IDE). Estructura estándar: src/main/java (clases Java), src/main/webapp (JSP, HTML, CSS, JS) y src/main/webapp/WEB-INF (web.xml, JSP privados). La URL queda http://localhost:8080/NOMBRE_DEL_WAR/pagina.jsp.

Código de las clases, explicado línea por línea

Cada recuadro gris reproduce líneas reales del archivo (con su número de línea) y el recuadro verde de abajo explica qué hacen y por qué están escritas así.

Clase 019 - 26-08: primer JSP con scriptlets y procesamiento de un formulario

Se crea una aplicación web Maven (**war**) con tres páginas: **index.jsp** (saludo según la hora del servidor, con scriptlets y expresiones), **formulario.html** (formulario estático de nombre y edad) y **procesar-form.jsp** (lee los datos del formulario y dice si es mayor de edad).

019 - Clase 26-08 / pom.xml

Qué es y para qué sirve: Archivo de configuración de Maven (POM) de una aplicación web: coordenadas, dependencias y nombre del WAR.

Cómo defenderlo / qué decir: Es el POM que genera el arquetipo de Maven para aplicaciones web. Lo importante: **packaging war**, las APIs de Servlet/JSP como **provided** (las aporta el contenedor) y, si la página usa JSTL, esas librerías sí van en el WAR.

```
1 | <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2 |   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
```

Líneas 1 a 2: Cabecera del POM: elemento raíz **project** con sus espacios de nombres XML (definen el formato válido del archivo).

```
3 |   <modelVersion>4.0.0</modelVersion>
```

Línea 3: Versión del modelo de POM (siempre 4.0.0).

```
4 |   <groupId>edu.usal</groupId>
5 |   <artifactId>clase-26-08</artifactId>
6 |   <packaging>war</packaging>
7 |   <version>1.0-SNAPSHOT</version>
8 |   <name>clase-26-08 Maven Webapp</name>
9 |   <url>http://maven.apache.org</url>
```

Líneas 4 a 9: Coordenadas del proyecto: **groupId** edu.usal (la organización), **artifactId** clase-26-08 (el nombre), **packaging war** (se empaqueta como aplicación web, un archivo .war que se despliega en Tomcat), versión 1.0-SNAPSHOT, nombre descriptivo y URL (datos del arquetipo maven-archetype-webapp).

```
10 |   <dependencies>
```

Línea 10: Abre la lista de dependencias (librerías que Maven descarga solo).

```
11 |     <dependency>
12 |       <groupId>junit</groupId>
13 |       <artifactId>junit</artifactId>
14 |       <version>3.8.1</version>
15 |       <scope>test</scope>
16 |     </dependency>
17 |
```

Líneas 11 a 17: JUnit 3.8.1 con **scope test**: solo se usa para pruebas unitarias, no viaja dentro del WAR.

```
18 |     <dependency>
19 |       <groupId>javax.servlet</groupId>
20 |       <artifactId>javax.servlet-api</artifactId>
21 |       <version>4.0.1</version>
22 |       <scope>provided</scope>
23 |     </dependency>
24 |
```

Líneas 18 a 24: javax.servlet-api 4.0.1 con **scope provided**: se necesita para compilar los servlets, pero en ejecución la provee el servidor (Tomcat 9), por eso no se incluye en el WAR.

```

25 | <dependency>
26 |   <groupId>javax.servlet.jsp</groupId>
27 |   <artifactId>javax.servlet.jsp-api</artifactId>
28 |   <version>2.3.3</version>
29 |   <scope>provided</scope>
30 | </dependency>

```

Líneas 25 a 30: javax.servlet.jsp-api 2.3.3, también **provided**: la API de JSP que aporta Tomcat.

```

31 | </dependencies>

```

Línea 31: Cierre de la lista de dependencias.

```

32 | <build>
33 |   <finalName>clase-26-08</finalName>
34 | </build>

```

Líneas 32 a 34: finalName define el nombre del WAR generado (clase-26-08.war) y por lo tanto la **ruta de contexto** de la aplicación: http://localhost:8080/clase-26-08/.

```

35 | </project>

```

Línea 35: Cierre del POM.

019 - Clase 26-08 / src / main / webapp / WEB-INF / web.xml

Qué es y para qué sirve: web.xml: descriptor de despliegue de la aplicación web. Vive en WEB-INF y es el archivo de configuración que lee el contenedor (Tomcat) al iniciar.

Cómo defenderlo / qué decir: Aquí solo contiene el nombre de la aplicación porque todavía no hay servlets declarados por XML. Es el lugar donde se declararían servlets, filtros, páginas de error y configuración de sesión (ver Tema 08).

```

1 | <!DOCTYPE web-app PUBLIC
2 |   "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3 |   "http://java.sun.com/dtd/web-app_2_3.dtd" >
4 |

```

Líneas 1 a 4: Declaración del tipo de documento (**DOCTYPE**): indica que es un descriptor de despliegue de aplicación web.

```

5 | <web-app>

```

Línea 5: Elemento raíz **web-app** en su versión mínima (Servlet 2.3): no declara espacios de nombres ni versión.

```

6 |   <display-name>Archetype Created Web Application</display-name>

```

Línea 6: Nombre descriptivo de la aplicación (lo muestra el administrador de Tomcat).

```

7 | </web-app>

```

Línea 7: Cierre del descriptor.

019 - Clase 26-08 / src / main / webapp / index.jsp

Qué es y para qué sirve: Página JSP que muestra un saludo («Buenos días / tardes / noches») y la hora del servidor, con un color distinto según el momento del día.

Cómo defenderlo / qué decir: Demuestra los elementos de JSP: **directivas** (page import), **scriptlet** con lógica, **expresiones** para imprimir variables y los dos **comentarios** (JSP vs HTML). Importante: la hora que se muestra es la **del servidor**, calculada cada vez que se pide la página. Para el parcial hay que decir que esta mezcla de Java y HTML es lo que luego se evita con JSTL.

```

1 | <%@ page import="java.time.LocalTime" %>
2 | <%@ page import="java.time.format.DateTimeFormatter" %>

```

Líneas 1 a 2: Directivas page import: importan LocalTime y DateTimeFormatter para poder usarlas en el código Java de la página (es el equivalente a los import de una clase).

```

3 | <html>
4 | <head>
5 |   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
   |     integrity="sha384-sRl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
6 | </head>
7 | <body>
8 |

```

Líneas 3 a 8: HTML de la cabecera: etiqueta html, head con el **link a Bootstrap** (CDN) y apertura del body.

```

9 | <%
10 |   //Scriptlets
11 |   LocalTime horaActual = LocalTime.now();
12 |   int hora = horaActual.getHour();
13 |   String msjSaludo;
14 |   String classColor;
15 |

```

Líneas 9 a 15: Scriptlet (<% ... %>): código Java que se ejecuta en el servidor. Obtiene la hora actual con LocalTime.now() y su hora (0-23). Declara las variables msjSaludo (texto) y classColor (clase CSS de Bootstrap).

```

16 |   if(hora >= 6 && hora < 12){
17 |       msjSaludo = "Buenos dias";
18 |       classColor = "alert-warning";
19 |   } else if(hora >= 12 && hora < 20) {
20 |       msjSaludo = "Buenas tardes";
21 |       classColor = "alert-info";
22 |   } else {
23 |       msjSaludo = "Buenas noches";
24 |       classColor = "alert-dark";
25 |   }
26 | %>
27 |

```

Líneas 16 a 27: Lógica de decisión: de 6 a 11 «Buenos días» con estilo amarillo (alert-warning); de 12 a 19 «Buenas tardes» con azul (alert-info); el resto «Buenas noches» con gris oscuro (alert-dark). El %> cierra el scriptlet.

```

28 |   <div class="container">
29 |       <!-- Comentario JSP --%>
30 |       <!-- Comentario HTML -->
31 |       <h2>JSP Clase 26-08</h2>
32 |

```

Líneas 28 a 32: Contenedor de Bootstrap. Dos comentarios: <!-- --%> (JSP: no llega al navegador) y <!-- --> (HTML: sí se envía, se ve en «ver código fuente»). Título h2.

```

33 |       <div class="alert <%= classColor %>">
34 |           <%= msjSaludo %>
35 |       </div>
36 |

```

Líneas 33 a 36: Expresión (<%= ... %>): imprime el valor de classColor dentro del atributo class, y en la línea siguiente imprime el saludo. Así el HTML final queda, por ejemplo, class="alert alert-info".

```

37 |       <p>
38 |           La hora del servidor es :
39 |           <strong>
40 |               <%= horaActual.format(DateTimeFormatter.ofPattern("HH:mm:ss"))%>
41 |           </strong>
42 |       </p>
43 |   </div>
44 | </body>
45 | </html>

```

Líneas 37 a 45: Muestra la hora del servidor formateada con **DateTimeFormatter.ofPattern("HH:mm:ss")**, mediante otra expresión. Cierra el contenedor, body y html.

019 - Clase 26-08 / src / main / webapp / formulario.html

Qué es y para qué sirve: Formulario HTML estático (página 1) que pide nombre y edad y los envía por POST a procesar-form.jsp.

Cómo defenderlo / qué decir: Muestra la conexión entre HTML y JSP: **method="post"** + **action="procesar-form.jsp"** y los atributos **name** (txtNombre, txtEdad), que son exactamente las claves que el JSP leerá con getParameter. Sin name los datos no viajan.

```

1 | <!DOCTYPE html>
2 | <html lang="en">
3 | <head>
4 |     <meta charset="UTF-8">
5 |     <title>Introduccion JSP</title>
6 |     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
7 |         integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">

```

Líneas 1 a 7: DOCTYPE, html, **head** con charset UTF-8, título y link a Bootstrap.

```

8 | <body>
9 |     <h2>Pagina 1 : Formulario HTML</h2>
10 |
11 |     <div class="container">

```

Líneas 8 a 11: Cuerpo: título «Página 1: Formulario HTML» y contenedor Bootstrap.

```

12 |         <form method="post"
13 |             action="procesar-form.jsp">
14 |             <div class="mb-3">

```

Líneas 12 a 14: **form**: el método POST y la **action** con la página que procesa los datos.

```

15 |                 <label class="form-label"
16 |                     for="inNombre">Ingrese su nombre :</label>
17 |                 <input class="form-control"
18 |                     id="inNombre"
19 |                     name="txtNombre">
20 |             </div>
21 |
22 |         <div class="mb-3">

```

Líneas 15 a 22: Campo de **nombre**: id="inNombre" (lo usa el label) y **name="txtNombre"** (el nombre del parámetro que viaja al servidor).

```

23 |                 <label class="form-label"
24 |                     for="inEdad">Ingrese su edad :</label>
25 |                 <input class="form-control"
26 |                     id="inEdad"
27 |                     type="number"
28 |                     name="txtEdad">
29 |             </div>
30 |
31 |         </div>

```

Líneas 23 a 31: Campo de **edad** de type="number" con **name="txtEdad"**.

```

32 |             <button type="submit" class="btn btn-primary">Enviar a procesar</button>
33 |         </form>
34 |     </div>
35 |
36 | </body>
37 | </html>

```

Líneas 32 a 37: Botón que envía el formulario.

019 - Clase 26-08 / src / main / webapp / procesar-form.jsp

Qué es y para qué sirve: Página JSP (página 2) que **recibe y procesa** los datos del formulario y muestra si la persona es mayor o menor de edad.

Cómo defenderlo / qué decir: Es el ejemplo de cómo un JSP lee un formulario: **request.getParameter**, convierte con **parseInt**, decide con un **if** y escribe con **out.println** y con una expresión. **Errores que conviene reconocer:** (1) la condición txtEdad != null || !txtEdad.isEmpty() debería usar **&&**: con || si el valor es null se produce un NullPointerException al evaluar el segundo término, y si llega vacío ("") igual se intenta parseInt y falla con NumberFormatException; (2) mostrar txtNombre sin escapar permite inyección de HTML/JS (XSS). Con JSTL se usa **<c:out>** para escapar.

```

1 | <%--
2 |   Created by IntelliJ IDEA.
3 |   User: usal
4 |   Date: 26/8/26
5 |   Time: 8:31 p. m.
6 |   To change this template use File | Settings | File Templates.
7 | --%>

```

Líneas 1 a 7: Comentario JSP generado por el IDE (autor y fecha); no se envía al navegador.

```

8 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>

```

Línea 8: Directiva page: contentType="text/html; charset=UTF-8" define el tipo MIME y la codificación de la respuesta (tildes correctas); language="java" indica el lenguaje de los scriptlets.

```

9 | <html>
10 | <head>
11 |   <title>Introduccion JSP</title>
12 |   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
13 |     integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
14 | </head>
15 | <body>
16 |   <h2>Pagina 2 : Procesado de Formulario por JSP</h2>

```

Líneas 9 a 16: Apertura de html y head con título y Bootstrap. Título «Página 2: Procesado de formulario por JSP».

```

17 | <%

```

Línea 17: Scriptlet: lee los datos del formulario con el objeto implícito **request**.

```

18 |     String txtNombre = request.getParameter("txtNombre");
19 |     String txtEdad = request.getParameter("txtEdad");
20 |

```

Líneas 18 a 20: request.getParameter("txtNombre") y "txtEdad": siempre devuelven **String** (o null si el parámetro no vino). El nombre del parámetro es el **name** del input del formulario.

```

21 |     int edad = 0;
22 |     if(txtEdad != null || !txtEdad.isEmpty()){
23 |         edad = Integer.parseInt(txtEdad);
24 |     }
25 |
26 |     %>
27 |

```

Líneas 21 a 27: Variable entera con valor inicial 0 y conversión del texto a número con **Integer.parseInt** (ver observación en la defensa).

```

28 |     <h2>¡Hola, <%= txtNombre %> </h2>
29 |     <%--<h4>¡Hola, <%= out.print(txtNombre) %> </h4>--%>
30 |
31 |     <%

```

Líneas 28 a 31: Imprime con una expresión el nombre recibido. Debajo hay un comentario JSP con una línea inválida (out.print dentro de una expresión no compila).

```

32 |         if(edad >= 18) {
33 |             out.println("<p> Sos mayor edad </p>");
34 |         } else {
35 |             out.println("<p> Sos menor edad </p>");
36 |         }
37 |     %>
38 |

```

Líneas 32 a 38: Otro scriptlet: decide si es mayor de edad y escribe el párrafo con **out.println** (escribe directamente en la respuesta).

```

39 |     <a href="formulario.html" class="btn btn-primary">Volver al formulario</a>
40 | </body>
41 | </html>

```

Líneas 39 a 41: Enlace (estilo botón) para volver al formulario. Cierra body y html.

Resumen para estudiar

- HTTP: petición (método + cabeceras + cuerpo) y respuesta (estado + cabeceras + cuerpo). GET consulta, POST modifica. 200 OK, 302 redirección, 404 no encontrado, 500 error del servidor.
- JSP = HTML + Java en el servidor; el cliente solo recibe HTML. Internamente se convierte en un servlet.
- Código JSP: expresión `<%= %>`, scriptlet `<% %>`, declaración `<%! %>`, directiva `<%@ %>`, comentario `<%-- --%>`.
- Objetos implícitos: request, response, out, session, application, config, pageContext.
- Formulario → name del input = parámetro leído con `request.getParameter("name")` (devuelve String).
- Proyecto war: src/main/webapp (JSP/HTML) + WEB-INF/web.xml; las APIs de servlet/JSP son provided.

Preguntas típicas

¿Qué diferencia hay entre un scriptlet y una expresión?

El scriptlet (`<% %>`) ejecuta varias líneas de Java sin imprimir nada por sí mismo; la expresión (`<%= %>`) evalúa una sola expresión e imprime su valor en la página.

¿Por qué se dice que un JSP es un servlet?

Porque el contenedor lo traduce a una clase que extiende `HttpServlet`, la compila y la ejecuta; el HTML se escribe con `out.println` en el método de servicio.

¿Qué devuelve `request.getParameter` si el campo no existe?

Devuelve null; si existe pero está vacío devuelve una cadena vacía. Por eso hay que validar antes de convertir a número.